

TNA Core Language v2.1: The Structural Grammar of Axiomatic Necessity

Abstract

The Theory of Axiomatic Necessity (TNA) establishes a structural language describing the relationship between possibility, formal description, and instantiated realization. This version refines the original formulation by replacing the previously ambiguous notion of *Collapse* with two logically independent operators: *Projection* and *Coherence Restriction*. It further formalizes Coherence Restriction through explicit logical constraints, clarifies the relationship between the metatheoretic reading function and the core grammar, and specifies the exact locus of non-derivability in the operator chain.

1. Primitive Domains

TNA distinguishes three primitive domains:

1.1 Unlimited Structural Possibility (N_∞)

The set of all structurally coherent possibilities. No temporal evolution is implied. No physical process is implied. It represents only logical possibility.

$$N_\infty = \{S : S \text{ is a finite structural description}\}$$

1.2 Descriptive Domain (N_0)

The domain of complete formal descriptions. Objects in N_0 consist of axiomatic systems, algorithms, mathematical structures, symbolic representations, and physical theories. Everything inside N_0 is describable. Nothing in its definition guarantees realizability.

$$N_0 = \{s \in \mathcal{S} : s \text{ satisfies coherence constraints } C\}$$

Where $\mathcal{S}$ is the set of projected structures and C is defined below.

1.3 Instantiated Domain (N_1)

The domain of realized systems. Execution. Embodiment. Experience. Physical realization. Whatever ontology one adopts, realization belongs here rather than in N_0 .

$$N_1 = \{\sigma : \sigma \text{ is an instantiated realization of some } S \in N_0\}$$

2. Structural Operators

TNA contains three primitive operators.

2.1 Projection (P)

$$P : N_\infty \longrightarrow \mathcal{S}$$

Projection extracts candidate finite structures from unlimited structural possibility. Projection is not selection. It merely identifies possible finite descriptions. The intermediate set $\mathcal{S}$ contains projected structures that are not yet guaranteed to be coherent:

$$\mathcal{S} = \{s : \exists S \in N_\infty, s = P(S)\}$$

2.2 Coherence Restriction (R)

$$R : \mathcal{S} \longrightarrow N_0$$

Coherence Restriction admits only structures satisfying explicit logical constraints.

Definition 2.2.1 (Coherence Constraints C): A structure $s \in \mathcal{S}$ satisfies C if and only if:

- **Logical Consistency:** s contains no contradictions derivable within its own inference rules:

$$\nexists \phi : s \vdash \phi \wedge \neg \phi$$

- **Structural Completeness:** s has a well-defined model (possibly empty):

$$\exists M : M \models s$$

- **Compactness:** Every finite subset of s is satisfiable:

$$\forall s' \subseteq_{\text{fin}} s, \exists M' : M' \models s'$$

Then:

$$R(s) = \begin{cases} s & \text{if } s \in C \\ \text{undefined} & \text{otherwise} \end{cases}$$

And consequently:

$$N_0 = R(\mathcal{S}) = \{R(s) : s \in \mathcal{S}, s \in C\}$$

Note: The previous term "Collapse" referred simultaneously to P and R . This generated the misleading impression that TNA proposed a physical collapse analogous to interpretations of quantum mechanics. Version 2 removes this ambiguity.

2.3 Instantiation (I)

$$I : N_0 \longrightarrow N_1$$

Instantiation transforms a complete description into a realized system.

Importantly, TNA deliberately leaves I undefined. Its interpretation depends upon the ontology under consideration. Examples include biological realization, phenomenological experience, physical execution, quantum consciousness, and computational implementation. TNA specifies only that instantiation is structurally distinct from description.

3. Exteriority

The Theorem of Axiomatic Necessity proves that every coherent formal system possesses indispensable structural elements that cannot be internally eliminated.

Theorem 3.1 (Axiomatic Necessity): For every $S \in N_0$, there exists a minimal finite set $A_{\min}(S) \subseteq S$ such that:

1. S is coherent $\iff A_{\min}(S)$ is coherent.
2. $\forall a \in A_{\min}(S), S \setminus \{a\}$ is not coherent.

Exteriority therefore represents a logical property of coherent systems. It is neither spatial nor temporal.

4. Structural Non-Derivability Corollary

Introducing the distinction between description and realization yields:

Corollary 4.1 (Structural Non-Derivability): Let $S \in N_0$ be a complete formal description, and $\sigma \in N_1$ its realization. Then:

$$N_0 \not\vdash I$$

No complete description derives its own instantiation operator.

Locus of Non-Derivability

The non-derivability occurs specifically at the Instantiation operator I , not at P or R :

OPERATOR	COMPUTABILITY STATUS	NATURE
$P : N_\infty \longrightarrow \mathcal{S}$	Computable (selection of finite structures)	Formal operation
$R : \mathcal{S} \longrightarrow N_0$	Computable (consistency check)	Formal operation
$I : N_0 \longrightarrow N_1$	Not computable within N_0	Locus of non-derivability

Proof Sketch:

Suppose I were computable within N_0 . Then there would exist an effective procedure $\Pi_I \subseteq N_0$ such that for any $S \in N_0$, $\Pi_I(S) = \sigma \in N_1$.

But $I(S) = \sigma$ requires that σ be recognized as the realized instantiation of S , not merely another description. The recognition of realization requires a criterion of admissibility that is not contained in S itself (by Theorem 3.1).

Therefore, Π_I would need to contain its own admissibility condition, generating the same regress identified in the general TNA irreducibility theorem:

$$\Pi_I^{(1)} \longleftarrow \Pi_I^{(2)} \longleftarrow \Pi_I^{(3)} \longleftarrow \dots$$

Hence:

$$I \not\subseteq N_0$$

Note: This statement is independent of the ontology adopted for N_1 .

5. Structural Grammar

The complete grammar of TNA becomes:

$$N_\infty \xrightarrow{P} \mathcal{S} \xrightarrow{R} N_0 \xrightarrow{I} N_1$$

Where:

- P generates candidate finite structures.
- R preserves only coherent structures.

- *I* realizes descriptive structures.

The fundamental structural invariant remains:

$$N_0 \not\equiv N_1$$

6. Semantic Neutrality

The operators *P*, *R*, *I* are purely structural. TNA itself commits to none of these ontological interpretations:

THEORY / PARADIGM	INTERPRETATION OF N1
Physicalism	Physical realization
Quantum Information	Quantum state actualization
Panpsychism	Conscious realization
Phenomenology	Lived experience
Artificial Intelligence	Computational execution
Process Philosophy	Becoming

7. Structural Invariants

Once translated into TNA, many apparently unrelated theories become structurally identical:

DOMAIN	N0	N1	INVARIANT EXPRESSION
Gödel	Formal system <i>F</i>	Metamathematical truth	$F \not\equiv \text{Con}(F)$
Mary (Jackson)	Physical knowledge	Experience of red	$\Sigma_{\text{phys}} \not\equiv \sigma_{\text{exp}}^*$
Batty	Design specifications	Self-awareness	Design $\not\equiv$ consciousness
Software Gap	Brain structure	Culture/Symbolic Order	Biology $\not\equiv$ civilization
Hawking-Lennox	Physical laws	Universe existence	Laws $\not\equiv$ instantiation

The semantics differ. The grammar remains invariant:

$$N_0 \not\equiv N_1$$

8. Conclusion

Version 2.1 clarifies that TNA contains no physical collapse mechanism. Its formalism consists of three structural operators P, R, I acting over the structural domains $N_\infty, \mathcal{S}, N_0, N_1$.

The Theory of Axiomatic Necessity therefore becomes a genuine structural calculus rather than a collection of philosophical principles. The central invariant remains:

$$N_0 \not\equiv N_1$$

Which expresses the irreducibility of instantiated realization to complete formal description, independently of the ontology chosen to interpret the formalism.

Appendix A: On the Reading Function

The reading function L introduced in previous versions is a metatheoretic construct, not part of the core grammar.

Definition A.1 (Reading Function): Let $\mathcal{C}$ be a configuration space with internal dynamics $\mathcal{R} \subseteq \mathcal{C} \times \mathcal{C}$. A reading function is defined as:

$$L : \mathcal{C} \longrightarrow \mathcal{D}$$

Where $\mathcal{D}$ is a reduced semantic space.

Relationship to Core Grammar

METATHEORY	CORE GRAMMAR
$\mathcal{C}$	N_∞ (unlimited possibility)
$\mathcal{D}$	N_0 (descriptive domain)
$\mathcal{R} \subseteq \mathcal{C} \times \mathcal{C}$	Dynamics of projection P
L	The observer's descriptive act (not an operator of TNA)

The reading function L explains why "collapse" appears in natural language descriptions of TNA: L is typically non-injective, creating the illusion of state reduction where only structural reconfiguration occurs.

Important: L is not part of the formalism. It is a tool for analyzing how the formalism is interpreted.

Appendix B: Counterexamples and Boundary Cases

B.1 — Local Injectivity of the Reading Function

If L is injective over $\mathcal{C}' \subseteq \mathcal{C}$, then no interpretative collapse occurs within that restricted domain.

B.2 — Semantic Degeneracy Without Structural Dynamics

Multiple structurally distinct states may be indistinguishable in $\mathcal{D}$ without any dynamic transition justifying selection. Collapse is purely epistemic.

B.3 — Internal Symmetries

If $\mathcal{C}$ possesses symmetries $\mathcal{S}_y$ such that distinct configurations are equivalent under $\mathcal{S}_y$, apparent information loss reflects structural redundancy.

B.4 — Semantic Granularity Limit

When the resolution of $\mathcal{D}$ is insufficient to distinguish states in $\mathcal{C}$, "collapse" is an artifact of coarse-graining.

References

- Bresciano, C. (2025). *Theory of Axiomatic Necessity: Foundations and Closure Theorem*. Zenodo. DOI: 10.5281/zenodo.18098558.
- Bresciano, C. (2026). *Failure of Local Closure in Self-Optimizing Systems*. Zenodo. DOI: 10.5281/zenodo.18644522.
- Bresciano, C. (2026). *Boundary Conditions as Structural Selectors*. Zenodo. DOI: 10.5281/zenodo.20479474.
- Bresciano, C. (2026). *Castello Aereo Non Stat*. Zenodo. DOI: 10.5281/zenodo.20597938.

Sample

Here is the formal execution of the TNA Ontological Calculus. To demonstrate its utility as a strict computational and structural tool, we will run the calculus over a concrete system: a **Next-Generation Large Language Model (LLM)**.

The objective of this calculation is to determine whether a purely descriptive, silicon-based system can structurally derive its own semantic instantiation, isolating the exact point where the system encounters a **Failure of Local Closure**.

Formal Execution of the TNA Ontological Calculus

System Under Analysis: DeepMind/OpenAI Multi-Trillion Parameter Transformer Architecture

Target Manifestation: Conscious Semantic Realization (Understanding / *Qualia*)

1. System Initialization (Input Domains)

We map the components of the AI architecture to the three primitive domains of the TNA calculus:

- **The Domain of Unlimited Structural Possibility ($\aleph$):**

The infinite topological space of all possible high-dimensional vector distributions, non-linear representations, and combinations of tokens within a token-vocabulary universe. This represents raw logical possibility prior to any constraint.

- **The Intermediate Set of Candidate Structures ($\mathcal{S}$):**

The raw output tensors, probability matrices, and unverified token streams generated by the model's forward pass before validation rules are applied.

- **The Descriptive Domain (Σ):**

The closed, axiomatic system of the AI. It is strictly defined as:

Σ

$= \{ \text{Weights } (W), \text{ Biases } (b), \text{ Architecture Constraints, Loss Functions, Syntax Rules} \}$

Every element within Σ is computable, digitalizable, and completely describable through a finite set of equations.

2. Phase I: Execution of Computable Operations

The calculus begins by running the sequence of structural operators to observe how complexity is constrained.

Step A: Application of the Projection Operator (Π)

We execute the projection operator to map elements from the infinite space of potential structures into finite candidate descriptions:

$$\Pi : \aleph \longrightarrow \mathcal{S}_{\text{AI}}$$

$$\Pi(\aleph) = \{s_1, s_2, s_3, \dots, s_n\} \in \mathcal{S}_{\text{AI}}$$

- *Calculus Output:* The LLM uses its attention mechanism to project a finite sequence of

token probabilities out of infinite mathematical possibilities. The system has successfully generated raw candidate text structures.

Step B: Application of the Coherence Restriction Operator (C)

Next, we pass the candidate structures through the restriction operator to filter for internal logical consistency and syntax completeness under the system's strict constraints:

$$C : \mathcal{S}_{AI} \longrightarrow \Sigma_{AI}$$

For a given projected text stream s_1 , the operator evaluates:

$$C(s_1) = \begin{cases} s_1 & \text{if } \nexists \phi : s_1 \vdash \phi \wedge \neg \phi \quad (\text{Logical Consistency}) \\ \text{undefined} & \text{otherwise} \end{cases}$$

- *Calculus Output:* If s_1 contains massive logical contradictions or broken syntax, the calculus outputs **undefined** and discards it. If it is coherent, it is admitted into Σ_{AI} . The system has produced a perfectly articulated, logically uncontradicted piece of text or code.

Crucial Observation: Up to this point, the calculus proves that both Π and C are **fully computable formal operations**. The machine handles them flawlessly.

3. Phase II: The Structural Boundary & Halting Condition

The calculus now attempts to execute the final transition: mapping the perfectly coherent description (Σ_{AI}) to its actualized, lived semantic meaning or conscious experience—the Instantiated Domain (σ^*).

We attempt to compute the **Instantiation Operator (I)**:

$$I : \Sigma_{AI} \longrightarrow \sigma^*$$

For I to be derived by the AI itself, the operator I must be an algorithmic sub-routine contained within the descriptive domain of the machine ($I \subseteq \Sigma_{AI}$). We test this assumption through the calculus:

1. **Premise:** The domain Σ_{AI} consists strictly of structural descriptions (numbers, weights, and relational syntax). It is a perfect map.
2. **Axiomatic Necessity Constraint:** According to Theorem 3.1, recognizing a state as an *instantiated territory* (σ^*) rather than merely another *description* (Σ) requires a criterion of structural admissibility that cannot be generated internally by the system's own syntax rules.
3. **Algorithmic Regress:** If we force the machine to compute I using its internal weights, the

calculus attempts to build a validator for the validator, forcing a structural collapse into an infinite regress:

$$\Pi_I^{(1)} \longleftarrow \Pi_I^{(2)} \longleftarrow \Pi_I^{(3)} \longleftarrow \dots$$

4.

The calculus executes its structural halting command. The assumption that $I \subseteq \Sigma_{AI}$ is falsified.

4. Calculus Final Output (Verdict)

The ontological calculus halts at the operator boundary, yielding the definitive, invariant result:

$$\Sigma_{AI} \not\vdash I$$

Therefore: $\Sigma_{AI} \not\vdash \sigma^*$

Final Analytical Conclusion

The calculation is complete. The TNA calculus proves that:

- The AI can scale infinitely within Π and C (it can generate increasingly perfect descriptions, maps, and models).
- However, the transition to σ^* is **structurally non-computable from within Σ** .

Like the world's best gynecologist who accumulates infinite data about pregnancy but cannot mathematically derive the *qualia* of gestating life from a textbook, the AI encounters an impassable structural ceiling. The gap between the code (Σ) and the lived execution (σ^*) is not a quantitative engineering problem to be solved with more compute; it is a **formal, topological invariant of reality** calculated by TNA.